# Day 1 : Introduction + Basic Python

## Artificial Intelligence

Artificial Intelligence is a field of computer science focused on creating machines capable of performing tasks that typically require human intelligence, such as reasoning, learning, problem solving, perception, and decision making using data, algorithms, and computational models.

**Example**
A smartphone face recognition system identifies a person by analyzing facial features, comparing them with stored data, and allowing secure device unlocking automatically.

## Machine Learning

Machine Learning is a subset of artificial intelligence where computers learn patterns from data and improve their performance automatically without explicit programming by using algorithms, statistical models, and training datasets to make predictions or decisions.

**Example**
An email spam filter learns from labeled emails to classify incoming messages as spam or legitimate based on patterns in text, sender information, and previous user actions.

## Deep Learning

Deep Learning is a subset of machine learning that uses multilayer neural networks to automatically learn complex patterns from large datasets, enabling systems to perform tasks such as image recognition, speech processing, and natural language understanding.

**Example**
A medical imaging system analyzes thousands of X-ray images to detect lung disease by automatically learning visual patterns associated with abnormal or healthy conditions.

**Types of Machine Learning:**

## Supervised Learning

Supervised learning is a type of machine learning where a model learns from labeled data. The system studies input data and correct answers together, then learns patterns to predict the correct output for new data.

**Example**
A toddler learning fruit names. Parents show an apple and say “apple”. After seeing many examples, the child can correctly identify an apple.

## Unsupervised Learning

Unsupervised learning is a type of machine learning where the system learns from data without labels. The algorithm finds patterns, similarities, or groups in the data automatically without being told the correct answers.

**Example**
A toddler playing with toys. Without instructions, the child puts cars together and dolls together by noticing similarities between toys.

## Reinforcement Learning

Reinforcement learning is a type of machine learning where a system learns by trial and error. It receives rewards for correct actions and penalties for wrong actions, helping it learn the best decisions.

**Example**
A toddler learning to stack blocks. When blocks stay balanced, parents praise the child, encouraging them to repeat the successful stacking.

**Some real life examples**

## Recommendation Systems

Recommendation systems suggest products, movies, or music based on user preferences and past behavior. Platforms like **Netflix** and **Amazon** analyze what users watch or buy, then recommend similar content or products, helping users quickly find things they might enjoy or need.

## Email Spam Detection

Email services such as **Gmail** use machine learning to detect spam messages. The system studies patterns in emails like suspicious words, unknown senders, or unusual links, then automatically places unwanted messages into the spam folder to keep the inbox clean and safe.

## Face Recognition

Face recognition systems used in smartphones and security cameras analyze facial features like eyes, nose, and face shape. Devices such as **Apple Face ID** compare the scanned face with stored data to identify the correct person and unlock the device securely.

# Introduction to Python

**Python:**

**Python** is a simple and powerful programming language used to write computer programs. It is easy to read and understand, which makes it popular for beginners and professionals. Python is widely used in web development, data analysis, automation, and artificial intelligence because it supports many useful libraries and tools.

## Use of Python in AI

1. Python is used to build **machine learning models** that learn patterns from data.
2. Python helps process and analyze large datasets for **AI applications**.
3. Python libraries help create **deep learning models** for images and speech.
4. Python is used to develop **AI systems like chatbots and recommendation systems**.
5. Python helps visualize data and model results using graphs and charts.

**Basic Concepts of Python programming:**

### Variable

A variable is a named storage location used to store data in a program. It allows programmers to save values such as numbers or text and reuse them later. The stored value can also change during program execution.

### Example

In Python, `x = 10` stores the value 10 in variable x. Printing x will display the stored number.

### Data Types

In **Python**, data types define the kind of value a variable can store, such as numbers, text, or logical values. They help the program understand how data should be stored, processed, and used during execution.

### Common Data Types in Python

### Integer (int)

Stores whole numbers such as 1, 25, or -10. Commonly used for counting, indexing, and numerical calculations.

### Float

Stores decimal numbers like 3.14 or 2.5. Used in calculations involving measurements, percentages, and continuous values.

### String (str)

Stores text data such as names, sentences, or messages. Strings are written inside quotation marks.

**Boolean (bool)**
Stores logical values True or False. Used in conditions, comparisons, and decision-making within programs.

## Example Code:

```
# Integer example
age = 30
print("Age:", age)

# Float example
price = 19.99
print("Price:", price)

# String example
name = "Atir"
print("Name:", name)

# Boolean example
is_student = True
print("Is Student:", is_student)
```

```
Age: 30
Price: 19.99
Name: Atir
Is Student: True
```

## Print Statement:

The **print statement** in **Python** is used to display output on the screen. It helps programmers show text, numbers, or results of calculations so users can see information produced by the program.

```
print ("this is my first python code")
```

```
this is my first python code
```

**Taking user input:**

In **Python**, user input is taken using the `input()` function. It allows the program to receive data from the user through the keyboard and store it in a variable for processing.

```
num = int(input("Enter an integer number: "))

print("The integer you entered is:", num)
```

```
Enter an integer number:  5
The integer you entered is: 5
```

```
value = float(input("Enter a decimal number: "))

print("The float value you entered is:", value)
```

```
Enter a decimal number:  15.5
The float value you entered is: 15.5
```

```
name = input("enter your name")
print("your name is", name)
```

```
enter your name syed atir raza
your name is syed atir raza
```

**List:**

In **Python**, a list is a collection used to store multiple values in a single variable. Items can be numbers, text, or mixed data. Lists are ordered, changeable, and written inside square brackets.

**Uses of Lists**

- Store multiple related values in one variable.
- Easily access items using index numbers.
- Modify, add, or remove items when needed.

```
# List of fruits
fruits = ["apple", "banana", "mango"]

# List of numbers
numbers = [10, 20, 30, 40]

# List of student names
students = ["Ali", "Raza", "Atir"]

print("Fruits List:", fruits)
print("Numbers List:", numbers)
print("Students List:", students)
```

```
Fruits List: ['apple', 'banana', 'mango']
Numbers List: [10, 20, 30, 40]
Students List: ['Ali', 'Raza', 'Atir']
```

## Tuple in Python

A **tuple** is a collection of items stored in one variable. It is **ordered and cannot be changed** after creation. Tuples use **round brackets** `()`.

```
numbers = (10, 20, 30, 40)

print(numbers)
print(numbers[1])
```

## Set

A set stores multiple unique values in one variable. Duplicate values are automatically removed. Sets are unordered, so items do not have a fixed position. Curly brackets `{ }` are used.

```
numbers = {1, 2, 3, 3, 4, 5}
print(numbers)
```

## Dictionary

A dictionary stores data in key-value pairs. Each key has a related value. It helps organize information like a small database. Dictionaries also use curly brackets `{ }`.

```
student = {"name": "Ali", "age": 20, "grade": "A"}
print(student)
print(student["name"])
```

**Loops:**

In **Python**, a loop is used to repeat a block of code multiple times automatically. It helps perform the same task again and again without writing the code repeatedly. Common loops in Python are **for loop** and **while loop**

**While Loop**

In **Python**, a while loop repeatedly executes a block of code as long as a specified condition remains true. It is useful when the number of repetitions is unknown beforehand and stops once the condition becomes false.

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

```
1
2
3
4
5
```

### For Loop

In **Python**, a for loop is used to repeat a block of code a fixed number of times by iterating over a sequence like a list, string, or range. It's commonly used for known repetitions.

For loop syntax:

```
for variable in sequence:
    # code block to execute
```

- **for** → keyword to start the loop
- **variable** → takes the value of each item in the sequence, one by one
- **in** → connects the variable with the sequence
- **sequence** → a list, string, or range to iterate through
- **code block** → indented statements that execute each iteration

```
[11]: for i in range(5):
          print(i)
```

```
0
1
2
3
4
```

```
[12]: for i in range(1,6):
          print(i)
```

```
1
2
3
4
5
```

```
[ ]: # for i in range(1, 6): → Ye line kehti hai ke i ki value 1 se 5 tak le lo, 6 tak nahi.

     # print(i) → Har dafa jab loop chale, i ki value screen par print ho jayegi.

     #Ye code 1 se 5 tak numbers ko ek-ek karke print karta hai. Loop har number ke liye print(i) chalata hai aur phir agla number leta hai.
```

### `break` Statement

`break` is used to stop a loop immediately. When Python encounters `break`, the loop ends and the program continues with the next statement outside the loop.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

### `continue` Statement

`continue` is used to skip the current iteration of a loop. Python ignores the remaining code in that iteration and moves to the next iteration.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

### `enumerate()`

`enumerate()` is used to get both the **index number and value** of items in a list while looping. It helps track the position of elements easily.

```
fruits = ["apple", "banana", "mango"]

for index, value in enumerate(fruits):
    print(index, value)
```

## Conditions in Python

In **Python**, conditions let the program make decisions based on certain criteria. Using statements like if, elif, and else, Python checks whether a condition is True or False and executes different code blocks accordingly to control the program's flow.

### if

The `if` statement checks a condition. If the condition is **True**, the indented code block under it runs.

### elif

`elif` stands for "else if." It checks another condition if the previous `if` or `elif` was **False** and runs code if True.

**else**

`else` runs a code block when **all previous `if` and `elif` conditions are False**, providing a default action.

```
# Take age input and check category
age = int(input("Enter your age: "))

if age < 20:
    print("Teenager")
elif age > 20:
    print("Above 20")
else:
    print("You are 20 years old")
```

```
Enter your age:  13
Teenager
```

### Operators in Python

Operators are symbols in **Python** used to perform operations on variables and values. They help do **math calculations, compare values, or combine data** easily.

## Arithmetic Operators

Used for basic math operations.

- `+` → Addition
- `-` → Subtraction
- `*` → Multiplication
- `/` → Division
- `%` → Modulus (remainder)
- `**` → Power
- `//` → Floor division

```
a = 10
b = 3
print(a + b)  # 13
print(a % b)  # 1
```

```
13
1
```

## Comparison Operators

Used to compare values and return **True or False**.

- `==` → Equal
- `!=` → Not equal
- `>` → Greater than
- `<` → Less than
- `>=` → Greater or equal
- `<=` → Less or equal

```
x = 5
y = 10
print(x < y)
```

```
True
```

## Logical Operators

Used to combine conditions.

- `and` → True if both conditions are true
- `or` → True if any one condition is true
- `not` → Reverses True/False

```
# Example of logical operators

age = int (input("enter age"))
has_permission = True

# Using 'and'
if age >= 18 and has_permission:
    print("You can enter the park.")

# Using 'or'
if age < 18 or not has_permission:
    print("You cannot enter the park.")

# Using 'not'
if not has_permission:
    print("Permission denied.")
```

```
enter age 20
You can enter the park.
```

### Function in Python

A function in **Python** is a reusable block of code that performs a specific task. Functions help organize code, avoid repetition, and make programs easier to read, debug, and maintain.

### Uses of Functions

- Organize code into smaller, manageable parts.
- Reuse code without rewriting it multiple times.
- Make programs easier to read, debug, and maintain.

```
#General Syntax of a function in python


def function_name(parameters):
    # code block
    return value  # optional
```

### Syntax Explanation

- `def` → keyword to define a function
- `function_name` → name of the function
- `parameters` → optional values passed to the function
- `return` → optional output value

```
# Function to add two numbers
def add_numbers(a, b):
    return a + b

result = add_numbers(5, 3)
print("Sum:", result)
```

```
Sum: 8
```

```python
# Define functions for basic operations
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"
# Take user input
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Choose operation
print("Select operation: +, -, *, /")
operation = input("Enter operation: ")

# Perform calculation
if operation == '+':
    print("Result:", add(num1, num2))
elif operation == '-':
    print("Result:", subtract(num1, num2))
elif operation == '*':
    print("Result:", multiply(num1, num2))
elif operation == '/':
    print("Result:", divide(num1, num2))
else:
    print("Invalid operation")
```

**Built-in Functions:**

- **len()** – Returns the number of items in a list, string, or other collection.
- **max()** – Finds the largest number or item in a list or collection.
- **min()** – Finds the smallest number or item in a list or collection.
- **sum()** – Adds all numbers in a list and returns the total sum.
- **type()** – Shows the data type of a variable or value.
- **round()** – Rounds a floating-point number to the nearest whole number or given decimals.

```python
# Example list of numbers
numbers = [10, 20, 5, 30, 15]

# len() - number of items
print("Number of items:", len(numbers))

# max() - largest number
print("Largest number:", max(numbers))

# min() - smallest number
print("Smallest number:", min(numbers))

# sum() - total of numbers
print("Sum of numbers:", sum(numbers))

# type() - data type
print("Data type of numbers:", type(numbers))

# round() - rounding a float
num = 7.678
print("Rounded number:", round(num, 1))  # round to 1 decimal place
```

```
Number of items: 5
Largest number: 30
Smallest number: 5
Sum of numbers: 80
Data type of numbers: <class 'list'>
Rounded number: 7.7
```

## Library in Python

A library in **Python** is a collection of pre-written code, modules, and functions that help programmers perform tasks quickly without writing code from scratch. Libraries simplify tasks like math operations, data analysis, and AI development.

```
#general syntax to import a library
import library_name

"""
Example: import math → imports Python's math library.

You can also import specific functions: from library_name import function_name
"""
```

Common Libraries in Python for AI:

**NumPy:**

**NumPy** is a library used to work with numbers and arrays in Python. It helps store lists of numbers efficiently and perform calculations like addition, subtraction, multiplication, or finding averages very quickly. Think of it as a **super calculator for lists of numbers**.

**Pandas:**

**Pandas** helps work with **tables of data** in Python, like Excel sheets. You can sort, filter, analyze, and clean your data easily. It makes understanding and handling large datasets simple without writing complex code.

### Matplotlib:

**Matplotlib** is used to **draw charts and graphs** in Python. It turns numbers and data into visual pictures, like bar charts or line graphs, making it easier to understand patterns, trends, and comparisons.

**Scikit-learn:**

**Scikit-learn** is a library that helps build **machine learning models** in Python. It allows computers to learn from data and make predictions, like recognizing spam emails or recommending movies, without writing complicated algorithms from scratch.

```python
# Importing main Python libraries for AI/ML
import numpy as np         # For numerical calculations
import pandas as pd        # For data tables
import matplotlib.pyplot as plt  # For graphs and charts
from sklearn import datasets       # For machine learning tools

print("Libraries imported successfully!")


"""import numpy as np → NumPy ko short name np diya gaya.

import pandas as pd → Pandas ko short name pd diya gaya.

import matplotlib.pyplot as plt → Matplotlib ko plt ke naam se use karenge.

from sklearn import datasets → Scikit-learn ka datasets module import kiya"""
```

Tasks:

## Task 1: Variables and Data Types

Write a Python program to:

- Take **your name** and **age** as input from the user.
- Print a message like:

  ```
  Hello [name], you are [age] years old.
  ```

## Task 2: Simple Calculator Using Arithmetic Operators

Write a program to:

- Take **two numbers** from the user.
- Print **their sum, difference, product, and division**.

## Task 3: Working with Lists

Write a program to:

- Create a list of **5 fruits**.
- Print the **first fruit**, **last fruit**, and **total number of fruits** in the list.

## Task 4: Loops and Conditions

Write a program to:

- Take a number **n** from the user.
- Print all **even numbers from 1 to n**.
- Print a message if **n is less than 1**.

## Task 5: Simple Function

Write a Python function to:

- Take **two numbers** as input.
- Return their **sum**.
- Call the function and print the result.

**Task 6:** Write a program to convert 5 kilometers into miles

**Task 7: Write a program to find the square root of 25.**

**Task 8:** Write a program that prints numbers from 1 to 5 but stops when it reaches 4 using `break`.

**Task 9:** Write a program that prints numbers from 1 to 5 but skips number 3 using `continue`.

**Task 10:** Create a tuple of three colors and print the first color.

**Task 11:** Create a dictionary with name and age of a student and print the name.

**Task 12:** Create a list of three fruits and print the index and name using `enumerate()`.

**Task 13:** Create a set with some duplicate numbers and print the set.

**Solution:**

```python
#task 1 solution
# Take input from user
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Print message
print("Hello", name + ", you are", age, "years old
```

```
Enter your name:  atir
Enter your age:  25
Hello atir, you are 25 years old.
```

```python
# Take two numbers from user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Perform calculations
print("Sum:", num1 + num2)
print("Difference:", num1 - num2)
print("Product:", num1 * num2)

if num2 != 0:
    print("Division:", num1 / num2)
else:
    print("Division by zero is not allowed")
```

```
Enter first number:  4
Enter second number:  5
Sum: 9.0
Difference: -1.0
Product: 20.0
Division: 0.8
```

```python
# List of fruits
fruits = ["apple", "banana", "mango", "orange", "grapes"]

# Access first and last fruit
print("First fruit:", fruits[0])
print("Last fruit:", fruits[-1])
print("Total number of fruits:", len(fruits))
```

```
First fruit: apple
Last fruit: grapes
Total number of fruits: 5
```

```python
# Take number from user
n = int(input("Enter a number: "))

if n < 1:
    print("Number should be at least 1")
else:
    print("Even numbers from 1 to", n, ":")
    for i in range(1, n+1):
        if i % 2 == 0:
            print(i, end=" ")
```

```
Enter a number:  6
Even numbers from 1 to 6 :
2 4 6
```

```python
# Function to add two numbers
def add_numbers(a, b):
    return a + b

# Take input from user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Call the function and print result
result = add_numbers(num1, num2)
print("Sum:", result)
```

```
Enter first number:  4
Enter second number:  5
Sum: 9.0
```

```python
km = 5
miles = km * 0.621371
print("Miles =", miles)
```

```python
num = 25
sqrt = num ** 0.5
print("Square root =", sqrt)
```

```python
for i in range(1,6):
    if i == 4:
        break
    print(i)
```

```python
for i in range(1,6):
    if i == 3:
        continue
    print(i)
```

```python
colors = ("red", "blue", "green")
print(colors[0])
```

```python
student = {"name":"Ali", "age":20}
print(student["name"])
```

```python
fruits = ["apple", "banana", "mango"]

for i, fruit in enumerate(fruits):
    print(i, fruit)
```

```python
numbers = {1, 2, 2, 3, 4, 4, 5}
print(numbers)
```